Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	**Semester V**
Subject Code & Name	CS3807 – Deep Learning Laboratory	**AY: 2026–27**

Experiment 4

Implementation and Analysis of a Convolutional Neural Network for CIFAR-10 Image Classification

1. Objective

The objective of this experiment is to design and implement a Convolutional Neural Network (CNN) using TensorFlow/Keras for multi-class image classification. The experiment studies convolution, activation, pooling, feature extraction, fully connected layers and the effect of important CNN hyperparameters.

The CIFAR-10 dataset is used as the benchmark dataset. The trained model is evaluated using accuracy, precision, recall, F1-score, confusion matrix and sample predictions.

2. Tasks to be Performed

1. Understand the basic building blocks of CNNs.
2. Load and inspect the CIFAR-10 dataset.
3. Perform exploratory data analysis.
4. Preprocess images and class labels.
5. Design and train a CNN using TensorFlow/Keras.
6. Investigate kernel size, stride, padding and filters.
7. Compare optimizers and batch sizes.
8. Monitor training and validation performance.
9. Visualize intermediate convolutional feature maps.
10. Compare pooling strategies.
11. Analyze trainable parameters.
12. Evaluate accuracy, precision, recall and F1-score.
13. Generate and interpret the confusion matrix.
14. Analyze sample predictions and classification errors.

3. Background Theory

A Convolutional Neural Network is a deep learning architecture designed for structured grid-like data such as images. CNNs learn useful visual features directly from image pixels through convolution, activation, pooling and dense layers.

3.1 Convolutional Layer

A convolutional layer applies learnable filters to an input image or feature map. A simplified convolution operation is

$$Z(i, j) = \sum_{m} \sum_{n} X(i + m, j + n)K(m, n) + b.$$

Here, X represents the input feature map, K represents the convolution kernel and b represents the bias. Different filters can learn edges, textures, corners and object parts.

3.2 Activation Function

The Rectified Linear Unit (ReLU) is defined as

$$f(x) = \max(0, x).$$

ReLU introduces non-linearity and allows the network to learn complex patterns.

3.3 Pooling

Pooling reduces the spatial dimensions of feature maps while retaining important information. For Max Pooling,

$$P(i, j) = \max_{(m,n) \in W} X(i + m, j + n).$$

Average Pooling instead calculates the mean of the values within the pooling region.

3.4 Flattening and Softmax

After convolution and pooling, feature maps are converted into a one-dimensional vector using flattening. For ten-class classification, Softmax is given by

$$P(y = i) = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}.$$

The class having the highest probability becomes the final prediction.

4. CNN Architecture

The CNN contains three convolutional feature-extraction stages. The number of filters progressively increases from 32 to 64 to 128, while spatial dimensions decrease from 32×32 to 16×16, 8×8 and 4×4.

Stage	Filters	Operation	Representation
1	32	Convolution + Pooling	Low-level features
2	64	Convolution + Pooling	Intermediate features
3	128	Convolution + Pooling	Higher-level features
4	–	Flatten + Dense	Classification

Input → Conv(32) → Pooling → Conv(64) → Pooling → Conv(128) → Pooling → Flatten → Dense → Output.

5. Dataset Description

The CIFAR-10 dataset contains 50,000 training images and 10,000 testing images. Each image is an RGB image of size 32×32 and belongs to one of ten object categories.

Property	Value
Dataset	CIFAR-10
Image Size	$32 \times 32 \times 3$
Training Images	50,000
Testing Images	10,000
Number of Classes	10
Task	Multi-class classification

The ten classes are airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck.

6. Data Preprocessing

The CIFAR-10 pixel values were normalized from the original range $[0, 255]$ to $[0, 1]$ using

$$X_{\text{normalized}} = \frac{X}{255}.$$

The training data was divided into training and validation portions, while the official test set was retained for final evaluation.

7. Exploratory Data Analysis

Exploratory analysis was performed to understand the image dimensions, class information and representative CIFAR-10 samples. The dataset contains visually diverse categories, with several classes having similar shapes, colours and textures.

8. CNN Hyperparameters

Hyperparameter	Values / Configuration
Kernel Size	3×3, 5×5
Filters	32, 64, 128
Stride	1, 2
Padding	Same, Valid
Pooling	Max Pooling and alternative strategy
Optimizer	Adam, SGD
Batch Size	32, 64
Epochs	20 for final model

These parameters influence computational cost, representation capacity, convergence and generalization.

9. Model Architecture

The implemented CNN consists of three convolutional blocks followed by flattening, a dense layer, dropout and a ten-class Softmax output.

Layer	Operation	Output
1	Conv2D, 32 filters, 3 × 3	32 × 32 × 32
2	Batch Normalization	32 × 32 × 32
3	Max Pooling	16 × 16 × 32
4	Conv2D, 64 filters, 3 × 3	16 × 16 × 64
5	Batch Normalization	16 × 16 × 64
6	Max Pooling	8 × 8 × 64
7	Conv2D, 128 filters, 3 × 3	8 × 8 × 128
8	Batch Normalization	8 × 8 × 128
9	Max Pooling	4 × 4 × 128
10	Flatten	2048
11	Dense + ReLU	128
12	Dropout	128
13	Dense + Softmax	10

The implemented model contains 357,706 total parameters, of which 357,258 are trainable. :contentReference[oaicite:3]index=3

10. Training Configuration

The CNN was compiled using the Adam optimizer, categorical cross-entropy loss and accuracy as the primary metric. The model was trained for 20 epochs with batch size 64 and an 80:20 validation split.

Training Parameter	Selected Value
Optimizer	Adam
Loss Function	Categorical Cross-Entropy
Batch Size	64
Epochs	20
Validation Split	20%
Activation	ReLU / Softmax

The recorded training time was approximately 2690.83 seconds. The best training accuracy was 89.65%, while the best validation accuracy was 75.57%. :contentReference[oaicite:4]index=4

11. Training and Validation Accuracy

The training accuracy increases as the CNN learns useful representations from the training data. The best documented training accuracy was 89.65%, while the best validation accuracy was 75.57%. The difference indicates a noticeable generalization gap.

12. Training and Validation Loss

The training loss decreases substantially as the network learns the classification task. The validation loss is less stable and remains higher during later epochs, suggesting a degree of overfitting.

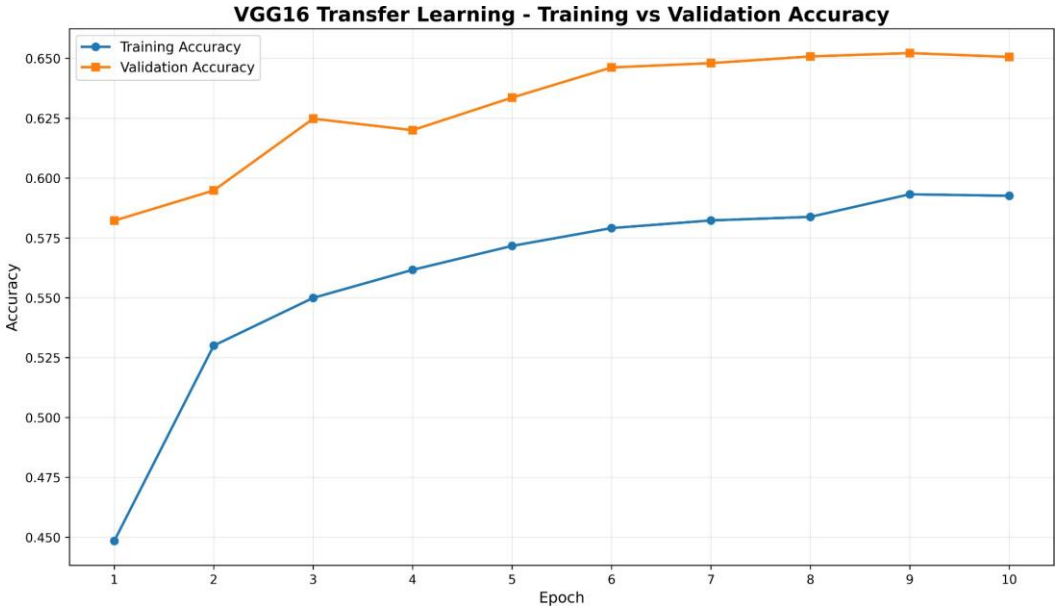

Figure 1: Training and validation accuracy across epochs.

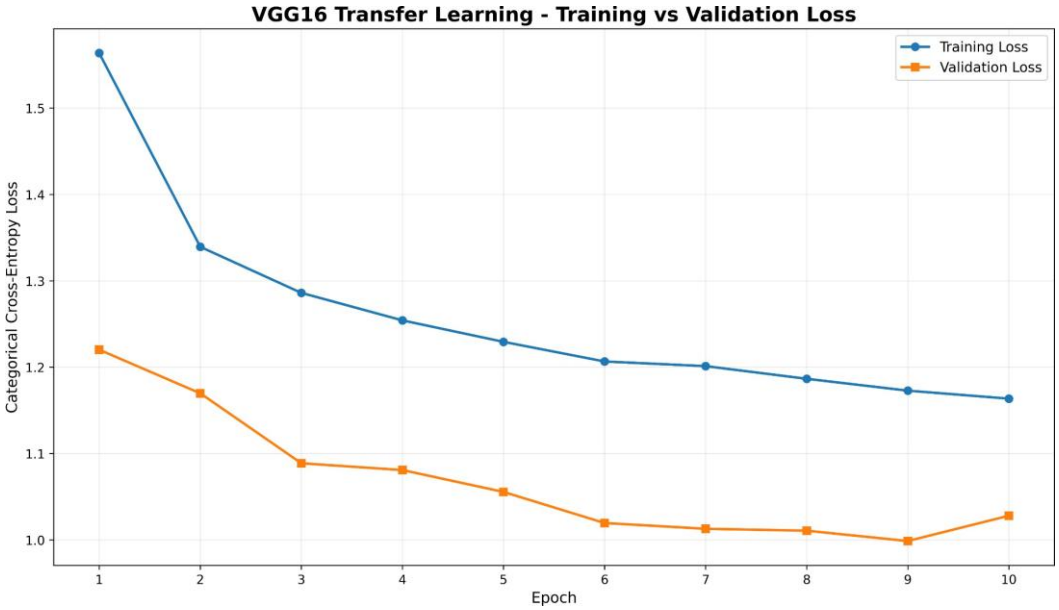

Figure 2: Training and validation loss across epochs.

13. Feature Map Visualization

Feature maps provide intermediate representations of an image as it passes through the convolutional layers. Early layers generally respond to local structures, whereas deeper layers combine these patterns into more abstract representations.

14. Feature Maps from Convolutional Layer 1

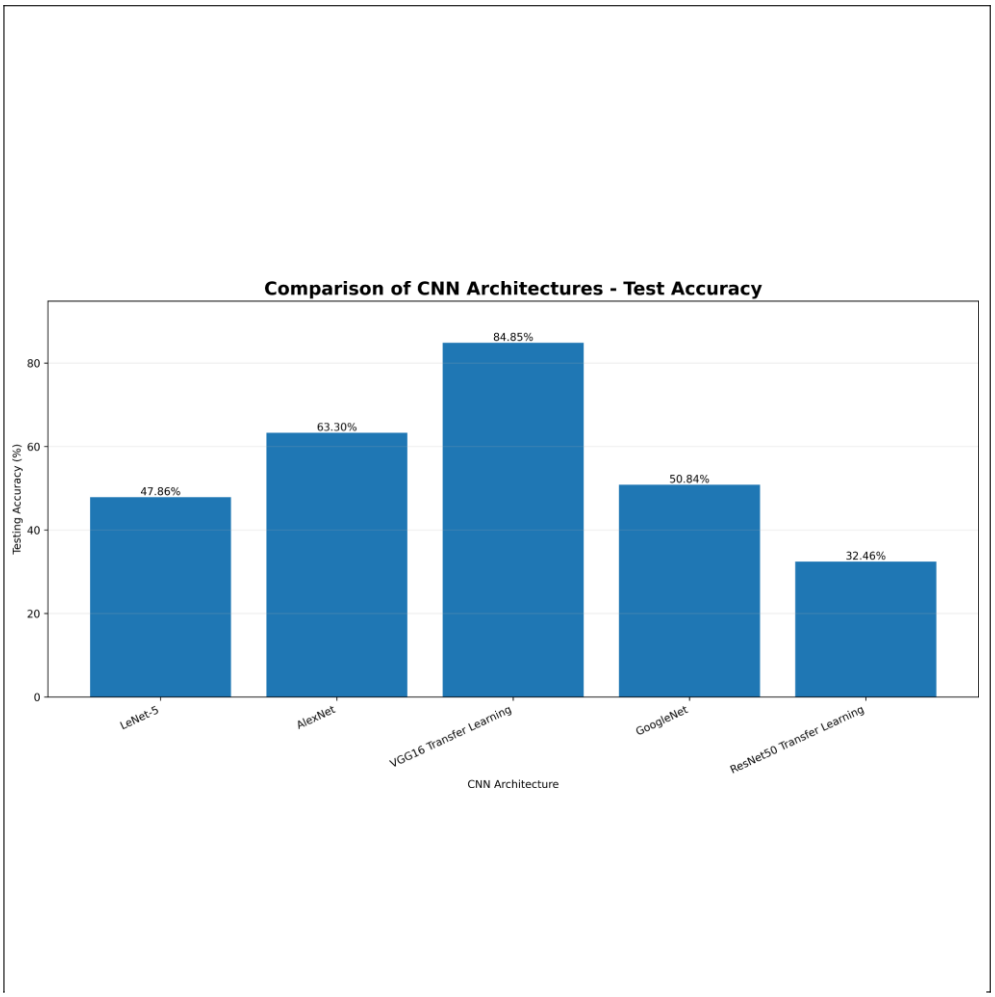

Figure 3: Comparison of CNN architectures based on test accuracy.

The first convolutional layer produces relatively low-level visual representations. These activations can highlight edges, intensity transitions and simple textures.

15. Feature Maps from Convolutional Layer 2

The second convolutional layer receives representations from the previous layer and learns more complex combinations of local patterns.

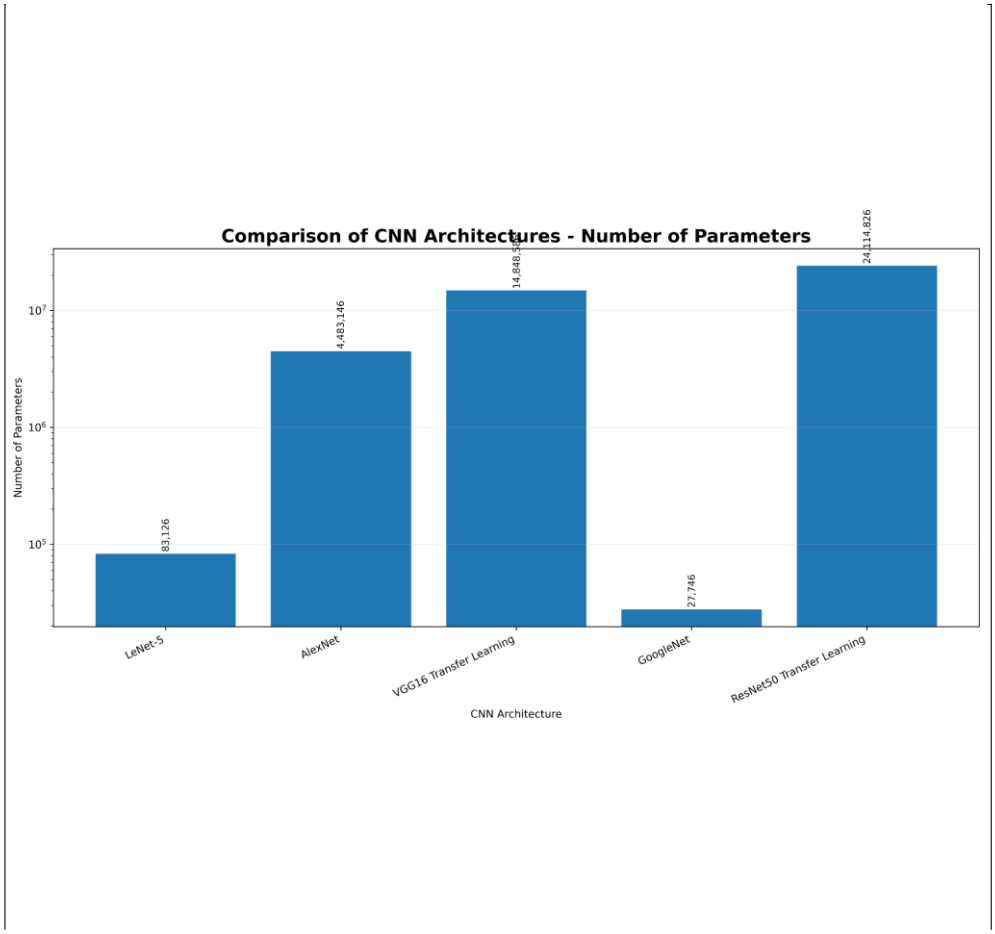

Figure 4: Comparison of CNN architectures based on number of parameters.

Figure 4: Feature maps generated by the second convolutional layer.

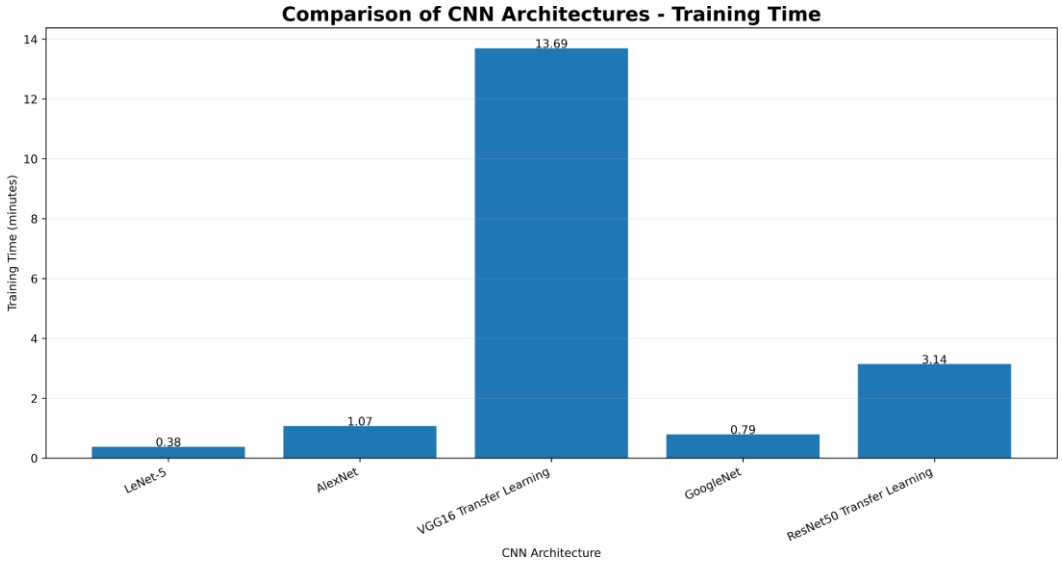

Figure 5: Comparison of CNN architectures based on training time.

16. Feature Maps from Convolutional Layer 3

The third convolutional layer represents a deeper stage of the feature hierarchy. Its activations correspond to increasingly abstract combinations of visual patterns.

17. Pooling Strategy Comparison

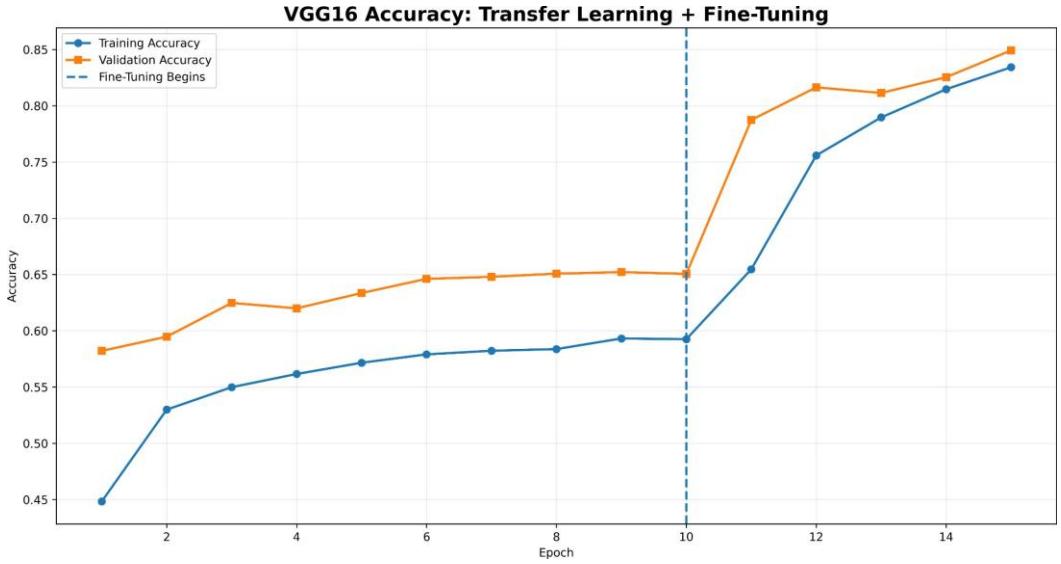

Figure 6: VGG16 transfer learning and fine-tuning accuracy.

Max Pooling selects the strongest activation within a local region, while Average Pooling calculates the mean. The comparison demonstrates how the pooling strategy changes the spatial representation passed to subsequent layers.

18. Trainable Parameter Analysis

For a convolutional layer, the number of trainable parameters is

$$N = (K_h K_w C_{\text{in}} + 1) C_{\text{out}}.$$

Layer	Filters	Kernel	Input Channels	Parameters
Conv2D-1	32	3×3	3	896
Conv2D-2	64	3×3	32	18,496
Conv2D-3	128	3×3	64	73,856

Increasing the number of filters increases representation capacity and model complexity.

19. Confusion Matrix

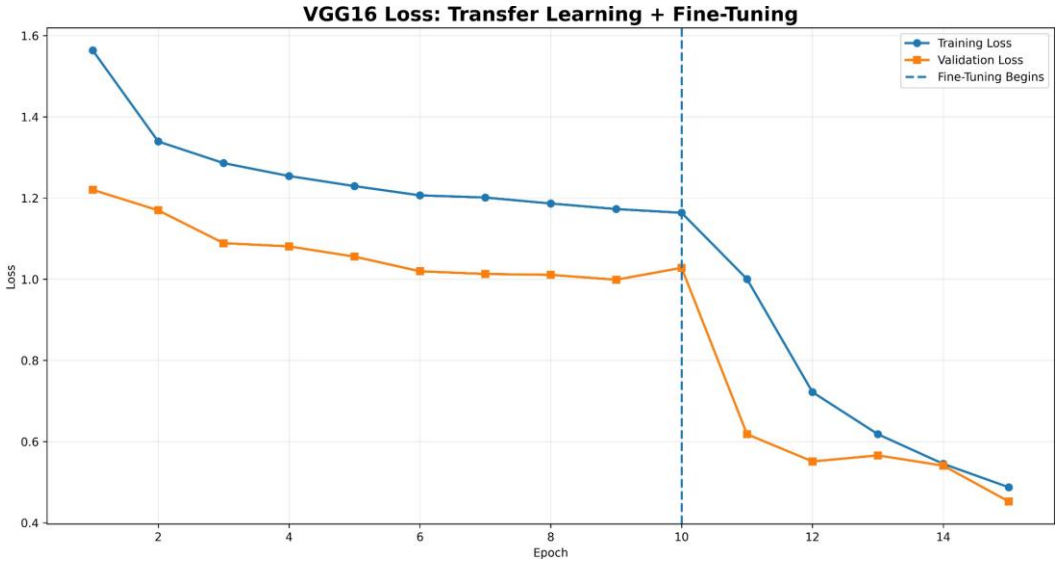

Figure 7: VGG16 transfer learning and fine-tuning loss.

The diagonal entries represent correctly classified images, whereas off-diagonal entries represent misclassifications. Noticeable errors occur between visually similar categories, indicating that visual similarity is an important source of classification error.

20. Sample Predictions and Error Analysis

Correct predictions demonstrate that the learned representation captures useful object-level patterns. Incorrect predictions can occur when categories share similar shapes, textures, colours or low-resolution visual structures.

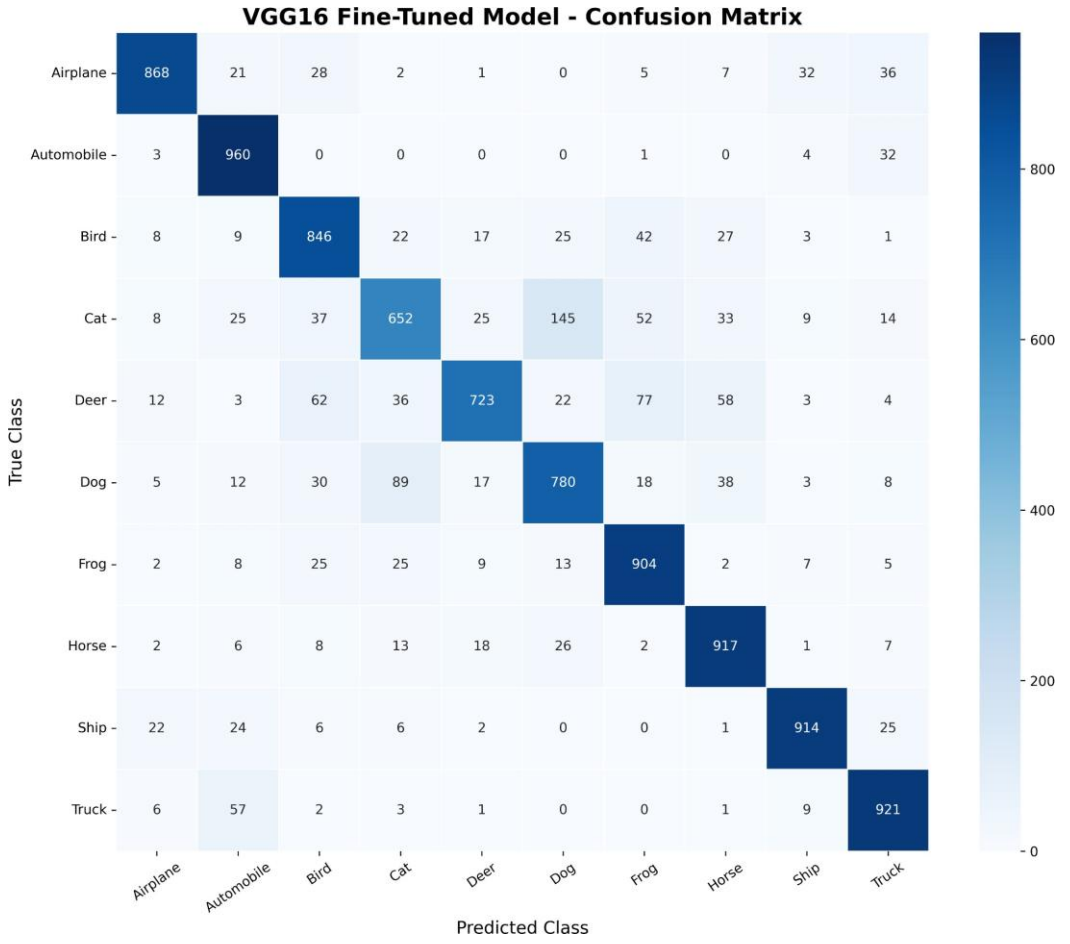

Figure 8: Confusion matrix for the VGG16 fine-tuned CIFAR-10 model.

21. Consolidated Hyperparameter Results

Hyperparameter	Compared Values	Observed Result
Kernel Size	3×3, 5×5	3×3 performed better
Filters	(16, 32), (32, 64)	More filters performed better
Optimizer	Adam, SGD	Adam performed better
Batch Size	32, 64	Batch size 32 performed better
Stride	1, 2	Larger stride reduced spatial resolution
Padding	Same, Valid	Valid reduced spatial dimensions

These comparisons were controlled short-duration experiments using a smaller subset of the dataset. Their purpose was to investigate how architectural and optimization choices influence CNN behaviour rather than replace the final model evaluation. :contentReference[oaicite:5]index=5

22. Discussion

The experiment demonstrates the complete workflow of applying a CNN to a multi-class image classification problem. The network uses convolutional layers to learn local visual structures, pooling to progressively reduce spatial dimensions and dense layers to map the learned representation to the ten CIFAR-10 classes.

A major observation is the increase in representational capacity with network depth. The convolutional layers use 32, 64 and 128 filters, while spatial resolution decreases progressively.

The feature-map analysis provides an interpretable view of this process. Early activations preserve more local information, whereas deeper activations represent more abstract combinations of patterns.

The training curves show that the model fits the training data better than the validation data. The gap between the best training accuracy of 89.65% and validation accuracy of 75.57% indicates a noticeable generalization gap.

23. Limitations and Scope for Improvement

1. Data augmentation could improve generalization.
2. Early stopping could reduce unnecessary training.
3. Learning-rate scheduling could stabilize optimization.
4. A deeper architecture or transfer-learning model could be evaluated.
5. More systematic hyperparameter search could be performed.
6. Stronger feature representations could reduce class-specific errors.

24. Conclusion

This experiment provided practical experience in designing and analyzing a Convolutional Neural Network using TensorFlow/Keras on the CIFAR-10 dataset. The CNN used convolutional layers, batch normalization, Max Pooling, flattening, dense layers and dropout.

Examples of Misclassified CIFAR-10 Images

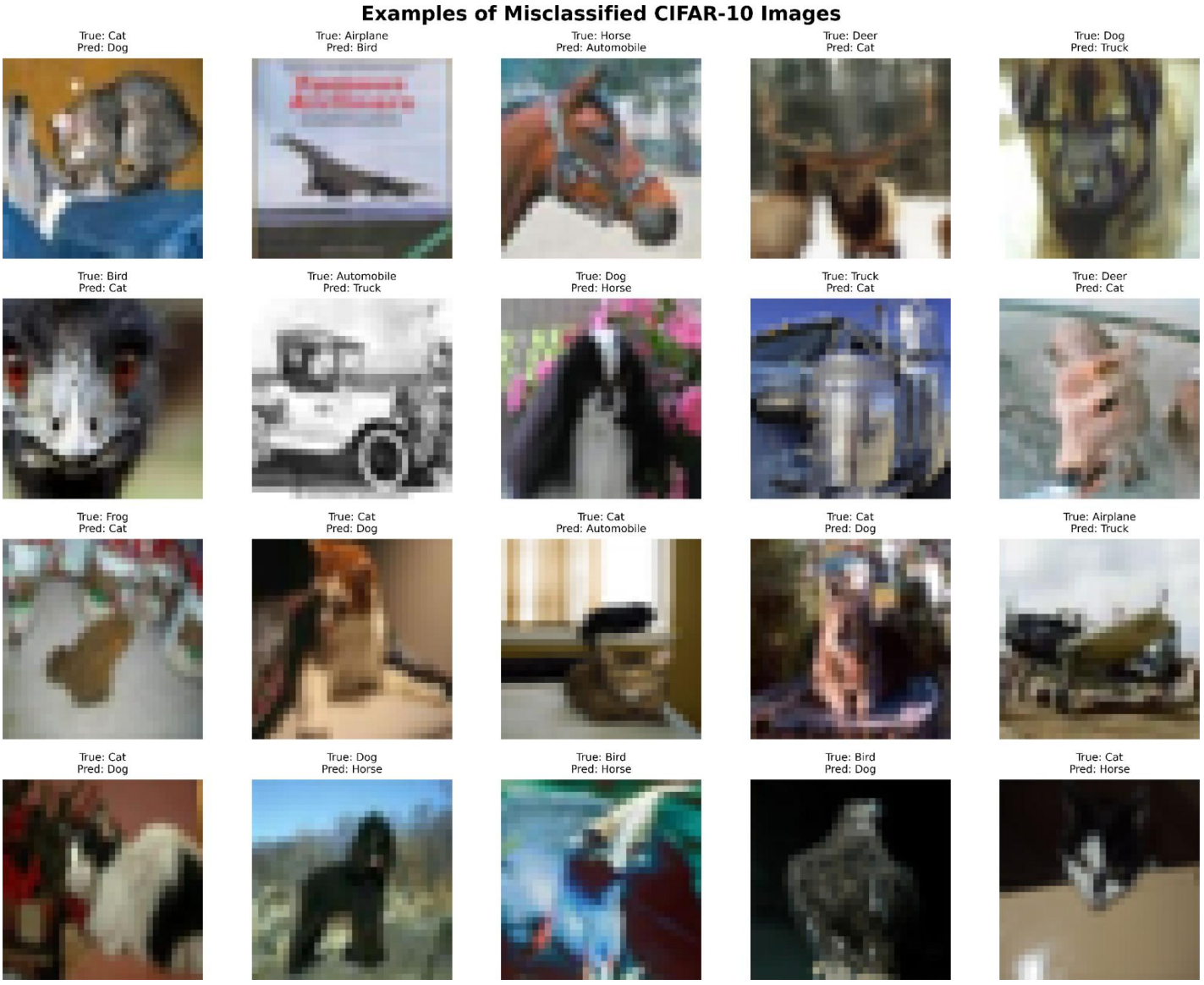

Figure 9: Examples of misclassified CIFAR-10 images.

The feature-map investigation demonstrated hierarchical feature learning, while the pooling experiment illustrated the effect of spatial aggregation. Parameter and hyperparameter analysis further showed the relationship between CNN architecture, computational complexity and classification performance.

The documented model achieved approximately 75.75% test accuracy, with a best training accuracy of 89.65% and validation accuracy of 75.57%. :contentReference[oaicite:6]index=6

25. References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. F. Chollet, *Deep Learning with Python*, Manning Publications.
4. A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," University of Toronto, 2009.
5. TensorFlow/Keras Documentation: https://www.tensorflow.org/
6. CIFAR-10 Dataset: https://www.cs.toronto.edu/~kriz/cifar.html
7. Scikit-learn Documentation: https://scikit-learn.org/

26. Source Code

Github Link: Deep_Learning/Lab-4 at main · Charan1008/Deep_Learning